

Document number	P3170R0
Date	2024-02-29
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG)

sinkable exception error message

Table of contents

- sinkable exception error message
 - Abstract
 - Motivational Example
 - Summary

Abstract

Currently there is an implication of superfluous dynamic allocation associated with C++ exception handling when dealing with informative and dynamic error messages. This paper proposes that we allow programmers the ability to avoid this cost wherever possible.

Motivational Example

Following is a common example of the superfluous dynamic allocations presented in this paper.

```

#include <string>
#include <vector>
#include <iostream>
#include <source_location>
#include <sstream>

using namespace std::literals::string_literals;

template<class T>
class my_vector : public std::vector<T> {
private:
    std::string to_string(const std::vector<T>::size_type pos, const std::source_location location) const
    {
        std::stringstream ss;
        ss << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") \""
            << location.function_name() << ": "
            << "invalid index: " << pos << ", size: " << this->size() << '\n';
        return ss.str();
    }

    [[nodiscard]]
    constexpr bool test( std::vector<T>::size_type pos ) const
    {
        return pos < this->size();
    }
public:
    using std::vector<T>::vector;
    typedef std::vector<T>::value_type value_type;
    typedef std::vector<T>::size_type size_type;
    typedef std::vector<T>::reference reference;
    typedef std::vector<T>::const_reference const_reference;
#if __STDC_HOSTED__ == 1
    [[nodiscard]]
    constexpr reference at( size_type pos, const std::source_location location = source_location::current() ) const
    {
        if(test(pos))
        {
            return (*this)[pos];
        }
        else
        {
            std::string temp = to_string(pos, location);
            throw std::out_of_range(temp); // POTENTIALLY COPYING OF std::string
        }
    }
    [[nodiscard]]

```

```

constexpr const_reference at( size_type pos, const std::source_location locati
{
    if(test(pos))
    {
        return (*this)[pos];
    }
    else
    {
        std::string temp = to_string(pos, location);
        throw std::out_of_range(temp);// POTENTIALLY COPYING OF std::string
    }
}
#endif
};

int main()
{
    try
    {
        my_vector<int> myints{1};
        int& pv = myints.at( 1 );//throws
    }
    catch(const std::out_of_range& oor)
    {
        std::clog << oor.what() << '\n';
    }
    return 0;
}

```

In the `my_vector` library author's code is a potentially superfluous dynamic allocation. This allocation is hidden from the library user who writes the `main` function. This allocation is a consequence of the `std::logic_error` and `std::runtime_error` along with their derived classes copying a string. This superfluous allocation frequently occurs when the error message is not static due to the varying values of the exception's members or due to internationalization.

```

logic_error( const std::string& what_arg );
invalid_argument( const std::string& what_arg );
domain_error( const std::string& what_arg );
length_error( const std::string& what_arg );
out_of_range( const std::string& what_arg );

runtime_error( const std::string& what_arg );
range_error( const std::string& what_arg );
overflow_error( const std::string& what_arg );
underflow_error( const std::string& what_arg );
system_error( std::error_code ec, const std::string& what_arg );
format_error( const std::string& what_arg );

```

This allocation could be mitigated by both the library author with the addition of sinkable move based overloads to the standard library.

```

logic_error( std::string&& what_arg );// sink the error message
invalid_argument( std::string&& what_arg );// sink the error message
domain_error( std::string&& what_arg );// sink the error message
length_error( std::string&& what_arg );// sink the error message
out_of_range( std::string&& what_arg );// sink the error message

runtime_error( std::string&& what_arg );// sink the error message
range_error( std::string&& what_arg );// sink the error message
overflow_error( std::string&& what_arg );// sink the error message
underflow_error( std::string&& what_arg );// sink the error message
system_error( std::error_code ec, std::string&& what_arg );// sink the error message
format_error( std::string&& what_arg );// sink the error message

```

In the original example, the second superfluous allocation could be avoided with a sinking move.

```

std::string temp = to_string(pos, location);
// POTENTIALLY COPYING OF std::string
throw std::out_of_range(temp);

```

```

std::string temp = to_string(
throw std::out_of_range(std::
// OR
throw std::out_of_range(to_st

```

Summary

With several extra overloads we can minimize dynamic allocations when performing exception handling. Instead of these superfluous allocations being hidden, they are placed in the control of exception throwers where it belongs.